Lecture 4: Kernel Regression Part 2

Adityanarayanan Radhakrishnan

November 12, 2025

1 Introduction

Kernel regression provides a natural extension of linear regression to nonlinear settings. Yet, when directly implementing kernel regression as presented in the previous lecture, we encounter an immediate limitation of this method: naive kernel regression implementations are slow for large datasets (super-quadratic in the number of samples). Below, we review techniques for drastically speeding up the runtime of kernel methods, making them practically relevant on large-scale datasets.

2 Speeding up kernels through finite dimensional feature maps

Recall the setup for kernel regression. We are given a kernel $K \in \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$, input data $X \in \mathbb{R}^{n \times d}$, and labels $y \in \mathbb{R}^n$. Let $K(X, X) \in \mathbb{R}^{n \times n}$ such that $K(X, X)_{ij} = K(x^{(i)}, x^{(j)})$. We solve for $\alpha \in \mathbb{R}^n$ such that $K\alpha = y$. Note that solving this linear system of equations as written requires super-quadratic time in the number of samples n. This approach is extremely fast if n is small (on the order of $10k$ samples or less).

Suppose now that n is large (on the order of hundreds of thousands or millions of samples). We run into two practical problems: (1) we cannot possibly materialize the matrix K in memory; and (2) super-quadratic runtime in the number of samples will take far too long (order of years or longer for such large datasets). Fortunately, there are standard techniques to overcome these practical limitations, making kernel methods practically feasible on large datasets.

The first such technique is straightforward if we are given an explicit feature map $\phi : \mathbb{R}^d \to \mathcal{H}$ for a kernel. Namely, we can use a finite dimensional truncation of the feature map ϕ and solve linear regression on feature-mapped data (as we did to motivate kernel regression in the previous lecture). Unfortunately, it is often nontrivial to find a feature map for many effective kernels (e.g., $\ell_{p,q}$ kernels of the form $K(x, z) = \exp\left(-L\|x - z\|_p^q\right)$).

Fortunately, for any kernel, there exists an approach called the *Nyström extension* that gives us a data-dependent feature map for any kernel. To develop the Nyström extension, we must first develop the notion of eigenfunctions of a kernel.

3 Kernel eigenfunctions and Mercer's theorem

The goal of this section is to prove *Mercer's theorem*, which decomposes kernels in terms of "eigenfunctions," an analog of eigenvectors from linear algebra for Hilbert spaces of functions. Building on the results of this section, we will ultimately approximate a kernel with a finite number of estimated eigenfunctions. This estimation naturally leads to the Nyström extension, one approach to scaling kernels to massive amounts of data. To get started, we first review helpful concepts from functional analysis below.

Analogously to the case of symmetric matrices in finite-dimensional vector spaces that have eigendecompositions, certain classes of *operators* (linear maps from a space to itself) on Hilbert spaces also have a corresponding notion of "eigendecomposition." Namely, we have the following spectral theorem for operators on Hilbert spaces:

Theorem 1 (Spectral theorem)**.** *Let $\mathcal{H}$ denote a Hilbert space with inner product $\langle \cdot, \cdot \rangle$. If $T : \mathcal{H} \to \mathcal{H}$ is a compact, self-adjoint operator, then there exists an orthonormal basis $\{e_i\}_{i=0}^{\infty} \subset \mathcal{H}$ of eigenvectors of T such that for any $f \in \mathcal{H}$,*

$$Tf = \sum_{k=0}^{\infty} \lambda_k \langle f, e_k \rangle e_k. \tag{1}$$

Here, the term orthonormal basis means that (1) $\mathcal{H} = \{\sum_{k=0}^{\infty} a_k e_k \ ; \ \{a_k\}_{k=0}^{\infty} \in \ell_2\}$; and (2) the $\{e_k\}$ satisfy

$$\langle e_j, e_k \rangle = \begin{cases} 0 & \text{if } j \neq k \\ 1 & \text{if } j = k \end{cases} . \tag{2}$$

The term eigenvector means that $Te_k = \lambda_k e_k$ for all k. We will apply the spectral theorem to the particular case of $\mathcal{H} = \mathcal{L}^2(\mu)$, where e_k are *eigenfunctions*.

Self-adjoint and compact operators. Before using the spectral theorem, we first have to define self-adjoint, compact operators. A *self-adjoint* operator, T, is one that satisfies $\langle Tf, g \rangle = \langle f, Ag \rangle$. An operator T is *compact* if for every bounded sequence $\{f_i\}_{i=1}^{\infty} \subset \mathcal{H}$, the sequence $\{Tf_i\}_{i=1}^{\infty} \subset \mathcal{H}$ has a convergent subsequence.

Kernel eigenfunction decomposition. In order to apply the spectral theorem and decompose kernels in terms of eigenfunctions, we need to first identify an appropriate Hilbert space and operator. Namely, we consider the following operator, $\mathcal{I}_K$, on the space $\mathcal{L}^2(\mu)$ of square integrable functions on a domain $\mathcal{X}$ with measure μ:

$$(\mathcal{I}_K f)(y) = \int_{\mathcal{X}} K(x, y) f(x) d\mu(x) := \langle K(\cdot, y), f \rangle, \tag{3}$$

where $K(\cdot, y) \in \mathcal{L}^2(\mu)$ for all $y \in \mathcal{X}$.

You will show in the homework that $\mathcal{I}_K$ is indeed a (bounded) operator on $\mathcal{L}^2(\mu)$. To apply the spectral theorem, we next need to show that $\mathcal{I}_K$ is self-adjoint and compact. Showing that $\mathcal{I}_K$ is a self-adjoint operator can be done by direct computation (you will do this in the homework). Showing that $\mathcal{I}_K$ is compact directly from the definition of compactness is nontrivial. Instead, we can use the following characterization of compact operators, which applies directly to our setting (a more general statement of the proposition below is presented in [1, 10.70]).

Proposition 1. *$\mathcal{I}_K$ in Eq. (3) is compact if $\int \int_{\mathcal{X} \times \mathcal{X}} |K(x, y)|^2 d\mu(x, y) < \infty$.*

Proposition 1 holds for kernels applied in several typical settings. For example, you will show in the homework that this proposition holds for any bounded kernel K when μ is a probability measure.

Returning to our derivation of eigenfunctions, consider a kernel K such that $\mathcal{I}_K$ is a self-adjoint, compact operator and so, the spectral theorem (Eq. (1)) holds (where $e_k \in \mathcal{L}^2(\mu)$ are functions). Let $f(\cdot) = K(\cdot, y) \in L^2(\mu)$. Then, $f(x) = \sum_{k=0}^{\infty} \langle f, e_k \rangle e_k(x)$ and

$$\langle f, e_k \rangle = \langle K(\cdot, y), e_k \rangle = (\mathcal{I}_K e_k)(y) = \lambda_k e_k(y), \tag{4}$$

where the last equality follows from the fact that e_k are eigenvectors for $\mathcal{I}_K$. Thus, we conclude

$$K(x, y) = f(x) = \sum_{k=0}^{\infty} \lambda_k e_k(x) e_k(y), \tag{5}$$

which yields the decomposition of K in terms of eigenfunctions.

4 Nyström extension

In the previous section, we decomposed a kernel K in terms of its eigenfunctions (Eq. (5)). The importance of this decomposition is that if we knew the eigenfunctions $\{e_k\}$ and eigenvalues $\{\lambda_k\}$, we could reconstruct the kernel $K(x, y)$ for *any* pair of data points x, y. In general, we do not have access to the data distribution μ and so, knowing all eigenfunctions and eigenvalues is impossible. The idea of the Nyström extension is to estimate finitely many such eigenfunctions and eigenvalues from a small subset of data points, and then use this finite subset to estimate the kernel at all other points.

Suppose for example that we given a finite number of data points $\{x^{(i)}\}_{i=1}^m$ where m is small ($m < 10k$). Given a kernel K, we wish to estimate eigenfunctions and eigenvalues for $\mathcal{I}_K$ from Eq. (3)based on these m data points. To this end, for any eigenvector e_j, we estimate $(\mathcal{I}_K e_j)(y)$ by replacing the integral in Eq. (3) with the finite sum

$$(\mathcal{I}_K e_j)(y) = \lambda_j e_j(y) \approx \frac{1}{m} \sum_{i=1}^m K(x^{(i)}, y) e_j(x^{(i)}). \tag{6}$$

Now suppose we set up Eq. (6) for each $y \in \{x^{(\ell)}\}_{\ell=1}^m$ and take the approximation as an equality. These m equations then form the system:

$$\frac{1}{m} \hat{K} \hat{E} = \hat{E} \hat{\Lambda}, \tag{7}$$

where

$$\hat{K} = \begin{bmatrix} K(x^{(1)}, x^{(1)}) & \dots & K(x^{(m)}, x^{(1)}) \\ \vdots & \dots & \vdots \\ K(x^{(1)}, x^{(m)}) & \dots & K(x^{(m)}, x^{(m)}) \end{bmatrix} ; \ \hat{E} = \begin{bmatrix} \hat{e}_1(x^{(1)}) & \dots & \hat{e}_m(x^{(1)}) \\ \vdots & \dots & \vdots \\ \hat{e}_1(x^{(m)}) & \dots & \hat{e}_m(x^{(m)}) \end{bmatrix} ; \ \hat{\Lambda} = \begin{bmatrix} \hat{\lambda}_1 & & \\ & \ddots & \\ & & \hat{\lambda}_m \end{bmatrix},$$

and $\hat{E}$ is an orthonormal matrix. On the other hand, note that $\hat{K}$ is a symmetric matrix and so $\hat{K} = Q \Lambda Q^T$ by eigendecomposition. We can thus use the eigendecomposition of $\hat{K}$ to derive an estimate for $\hat{E}$ and $\hat{\Lambda}$. Namely, accounting for the scale factor of $\frac{1}{m}$ in Eq. (7), we can set

$$\hat{e}_j(x^{(\ell)}) = \sqrt{m} Q_{j\ell} \ \ ; \ \ \hat{\lambda}_j = \frac{1}{m} \lambda_j \ . \tag{8}$$

Eq. (8) gives an approach to estimate eigenfunctions and eigenvalues from the eigenvectors and values of the empirical kernel matrix $\hat{K}$ computed on m samples.

Given these eigenfunctions, we can now estimate the value of the kernel at any point by using a finite dimensional version of our kernel eigenfunction equation (Eq. (7)):

$$\begin{aligned}
K(x, y) &= \sum_{i=1}^m \hat{\lambda}_i \hat{e}_i(x) \hat{e}_i(y) \\
&= \sum_{i=1}^m \frac{1}{\hat{\lambda}_i} \left(\hat{\lambda}_i \hat{e}_i(x) \right) \left(\hat{\lambda}_i \hat{e}_i(y) \right) \\
&= \sum_{i=1}^m \frac{1}{\hat{\lambda}_i} \left(\frac{1}{m} \sum_{j=1}^m K(x^{(j)}, x) \hat{e}_i(x^{(j)}) \right) \left(\frac{1}{m} \sum_{j=1}^m K(x^{(j)}, y) \hat{e}_i(x^{(j)}) \right) \quad \text{(Using Eq. (6))} \\
&= \sum_{i=1}^m \frac{1}{\lambda_i} \left(\sum_{j=1}^m K(x^{(j)}, x) Q_{ij} \right) \left(\sum_{j=1}^m K(x^{(j)}, y) Q_{ij} \right) \quad \text{(Using Eq. (8))} \\
&= \sum_{j,j'=1}^m K(x, x^{(j)}) \left(\sum_{i=1}^m \frac{1}{\lambda_i} Q_{ij} Q_{ij'} \right) K(x^{(j')}, y) \\
&= K(X, x)^T \hat{K}^{-1} K(X, y),
\end{aligned} \tag{9}$$

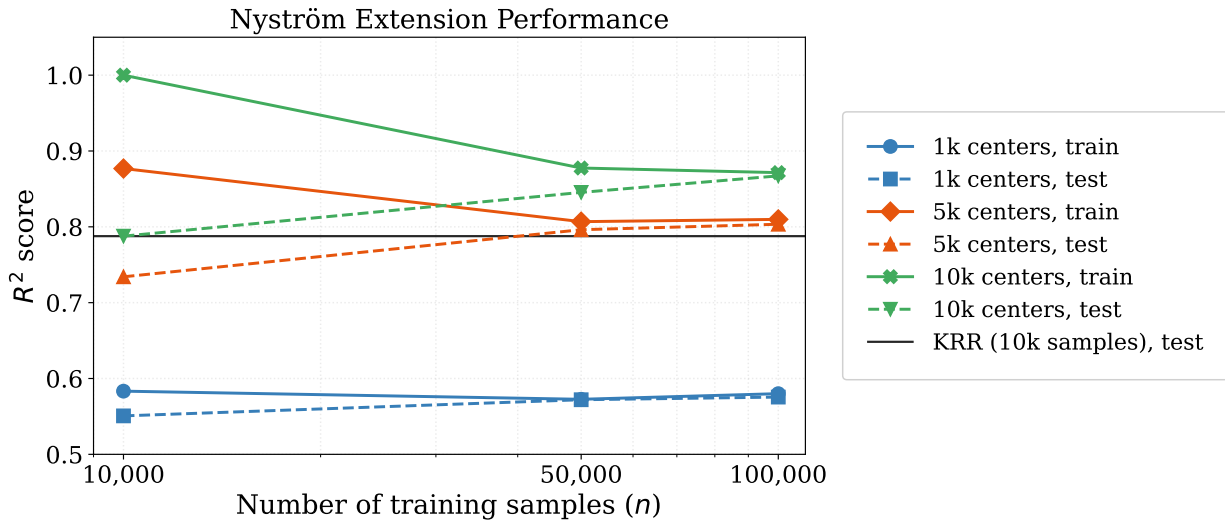

Figure 1

where $K(X, z) \in \mathbb{R}^m$ with $K(X, z)_i = K(x^{(i)}, z)$ for any $z \in \mathcal{X}$.

Eq. (9) is the Nyström technique for evaluating kernels at any point from a small, finite number of samples (commonly referred to as *centers*). We can use Eq. (9) to overcome the two computational limitations of kernel methods motivating this lecture.

Solving kernel regression efficiently with Nyström feature maps. Recall our motivating problem: we want to perform kernel regression with n samples $X' \in \mathbb{R}^{n \times d}$ where n is large (think hundreds of thousands to billions). We can first compute $\hat{K}^{-\frac{1}{2}}$ on a small number of $m = O(1)$ samples, X. Then, we can compute the matrix $\Phi = K(X', X)\hat{K}^{-\frac{1}{2}} \in \mathbb{R}^{n \times m}$, which requires $O(n)$ memory. We use the notation Φ to indicate that the resulting matrix is a finite-dimensional feature map approximation for K. Lastly, we can now perform *linear* regression on the feature mapped data Φ, which requires super-quadratic runtime in m.

5 Experiments

How useful is the Nyström extension in practice? As a preliminary evaluation, we compare the performance of the Nyström extension to full kernel regression on the synthetic data in the example below.

Example 1. *Let $x \sim \mathcal{N}(0, I_5)$ and let $y = \sin(x_1 x_2 x_5) + \cos(2x_3) + 5\sin(x_4)$. We first sample $n = 10,000$ training and test pairs from this data generating process and run kernel regression with the Laplace kernel $K(x, z) = \exp\left(-\frac{1}{L}\|x - z\|_2\right)$ with bandwidth $L = \sqrt{5}$ and a regularization parameter of $\lambda = 10^{-3}$. We next vary sample size $n \in \{10000, 50000, 100000\}$ and evaluate the performance of the Nyström extension using $m \in \{1000, 5000, 10000\}$ centers (Fig. 1). For all methods, we compute training and test R^2.*

We chose these ranges of centers to ensure that the number of parameters in the Nyström extension is at most the number of parameters in full kernel regression, which is 10000 in this setting. As a sanity check, we first find that when $n = m = 10000$, the performance of the Nyström extension exactly matches that of Kernel Ridge Regression (KRR). Next, we observe that Nyström extension matches or outperforms kernel ridge regression when the number of centers $m = 5000$ or 10000 and the number of samples $n > 50000$. Indeed, for the particular case of $m = 10000$ and $n = 100000$, we have a sizable gap in performance between Nyström and KRR (about .1 increase in test R^2). Moreover, with $n = 100,000$ samples, we could not use standard solvers to perform KRR even on an A100 GPU (80GB VRAM). On the other hand, the Nyström extension with $m = 10000$ centers on $n = 100000$ samples took roughly 1 minute (wall clock time).

6 Approximate kernel regression solvers

Thus far, we have introduced the Nyström extension as one approach for scaling kernels to massive datasets. Despite its effectiveness, the Nyström extension inherently utilizes a finite dimensional approximation of an infinite dimensional feature map. As such, when the number of samples increases well beyond the number of centers, our model becomes under-parameterized – it is not capable of achieving zero training error and training error is roughly equal to the test error. This phenomenon can be observed in Fig. 1 for all values of m (the number of centers).

Instead of speeding up kernel regression on large datasets by turning to smaller, approximate models, we can instead kept model size fixed and turned to *approximate solvers*. Indeed, rather than solving kernel regression exactly, we will approximately solve kernel regression via gradient descent. As we will see, the main challenge to overcome with gradient descent is that naively implementations will require a tremendous number of iterations to converge. To overcome this challenge, we will develop a preconditioned gradient descent algorithm (EigenPro) that converges in a handful of iterations and allows us to scale kernel-based approaches to millions of samples.

7 Gradient descent and Richardson iteration

Before turning to kernels with infinite dimensional feature maps, suppose we are given a kernel K of the form $K(x, z) = \langle \psi(x), \psi(z) \rangle$ where $\psi : \mathbb{R}^d \to \mathbb{R}^k$ is a finite dimensional feature map. To simplify notation, let $\psi(X) \in \mathbb{R}^{n \times k}$ denote the matrix arising from applying ψ to each row of $X \in \mathbb{R}^{n \times d}$. Suppose we use gradient descent with learning rate η to solve linear regression on feature-mapped data $(\psi(X), y)$. Gradient descent updates proceed as follows:

$$w_{t+1} = w_t - \eta \; \psi(X)^T \left(\psi(X) w_t - y \right). \tag{10}$$

How does this update correspond to updates in the *dual space* of kernel regression coefficients where we parameterize using $w_t := \psi(X)^T \alpha_t$? By direct substitution,

$$\begin{aligned}
&\psi(X)^T \alpha_{t+1} = \psi(X)^T \alpha_t - \eta \; \psi(X)^T \left(\psi(X) \psi(X)^T \alpha_t - y \right) \\
\implies &K \alpha_{t+1} = K \alpha_t - \eta \; K (K \alpha_t - y) \qquad \text{(Multiply by } \psi(X) \text{ and use } K = \psi(X)\psi(X)^T.) \\
\implies &\alpha_{t+1} = \alpha_t - \eta \; (K \alpha_t - y). \qquad (K \text{ is invertible.})
\end{aligned} \tag{11}$$

The update rule in Eq. (11) is called Richardson iteration. Note the distinction between Richardson iteration and applying gradient descent directly on the objective $\|K\alpha - y\|_2^2$: the latter uses an update rule of the form $\alpha_{t+1} = \alpha_t - \eta \; K^T (K \alpha_t - y)$. Indeed, Richardson iteration omits the extra K^T term and saves on computational cost.

With Eq. (11) in hand, we can simply run Richardson iteration for a number of steps to solve kernel regression. The question is then how many steps do we actually need to converge? Unfortunately, the answer is far too many steps. Returning to our example in Fig. 1, with $n = 10000$, we needed well over 10k steps of Richardson to achieve reasonable performance (see Fig. 2). We will overcome this practical limitation in the next section by applying *pre-conditioned gradient descent*.

8 Pre-conditioned gradient descent and Richardson iteration

The reason the updates in Eqs. (10) and (11) converge so slowly is that the spectrum of $H = \psi(X)^T \psi(X)$ (and correspondingly, that of K) decays sharply in practice and thus, these matrices are poorly conditioned (the ratio of the top eigenvalue to the smallest eigenvalue is large). Thus, even with an optimal choice of learning rate, we can converge well in the top eigen-direction of the data but then are converging slowly in the other directions. In order to speed up convergence of gradient descent and Richardson iteration, we *pre-condition* our updates so that convergence happens rapidly in the top k eigen-directions.

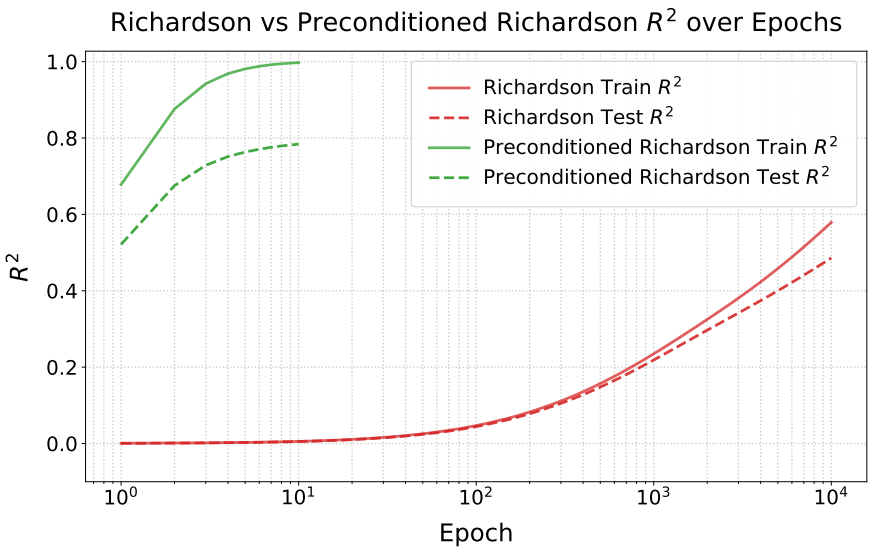

Figure 2

Namely, suppose that we precomputed the top k eigenvectors and values $\{(e_i, \lambda_i)\}_{i=1}^k$ of H. Then, we could better condition the matrix H by multiplying H with the matrix

$$\mathcal{P} = I_d - \left(\sum_{i=1}^{k} \left(1 - \frac{\lambda_{k+1}}{\lambda_i} \right) e_i e_i^T \right). \tag{12}$$

In particular, we have that e_i is still an eigenvector of $\mathcal{P}H$ with eigenvalue:

$$\lambda_i(\mathcal{P}H) = \lambda_i - \lambda_i + \lambda_{k+1} = \lambda_{k+1}. \tag{13}$$

By multiplying with $\mathcal{P}$, we have improved the conditioning along the top k eigen-directions by shrinking the top k eigenvalues down to λ_{k+1}. Hence, instead of performing vanilla gradient descent as in Eq. (10), we will use the update rule:

$$w_{t+1} = w_t - \eta\, \mathcal{P}\psi(X)^T(\psi(X)w_t - y). \tag{14}$$

How does this update rule translate to a corresponding Richardson update in the dual space? Following a similar computation as in the previous section, we have

$$\begin{aligned}
&\psi(X)^T\alpha_{t+1} = \psi(X)^T\alpha_t - \eta\, \mathcal{P}\psi(X)^T\left(\psi(X)\psi(X)^T\alpha_t - y\right) \\
\implies &K\alpha_{t+1} = K\alpha_t - \eta\, \psi(X)\mathcal{P}\psi(X)^T(K\alpha_t - y) \qquad \text{(Multiply by } \psi(X) \text{ and use } K = \psi(X)\psi(X)^T.) \\
\implies &\alpha_{t+1} = \alpha_t - \eta\, K^{-1}\psi(X)\mathcal{P}\psi(X)^T(K\alpha_t - y). \qquad (K \text{ is invertible.})
\end{aligned} \tag{15}$$

Eq. (15) is, as of yet, impractical because it still relies on knowledge of a feature map ψ. To resolve this issue and simplify further, we utilize the following relationship between the eigenvectors of $H = \psi(X)^T\psi(X)$ and K.

Lemma 1. *For any matrices $H = Z^TZ$ and $K = ZZ^T$, if e_i is an eigenvector of H with eigenvalue λ_i, then $\frac{1}{\sqrt{\lambda_i}}Ze_i$ is an eigenvector of K with eigenvalue λ_i.*

The proof of this lemma is left as an exercise (hint: remember that by convention eigenvectors are normalized to have an ℓ_2-norm of 1.) Applying Lemma 1 to Eq. (15) and using the formula for $\mathcal{P}$ from Eq. (12), we have

$$\begin{aligned}
\alpha_{t+1} &= \alpha_t - \eta \left(I_n - \left(\sum_{i=1}^{k} \left(1 - \frac{\lambda_{k+1}}{\lambda_i} \right) u_i u_i^T \right) \right) (K\alpha_t - y) \\
&= \alpha_t - \eta(K\alpha_t - y) + \eta B(K\alpha_t - y);
\end{aligned} \tag{16}$$

where $B = \sum_{i=1}^{k} \left(1 - \frac{\lambda_{k+1}}{\lambda_i}\right) u_i u_i^T$. Eq. (16) gives a pre-conditioned Richardson iteration update for solving kernel regression. Eq. (16) presents the fundamental idea and update rule of the *EigenPro* algorithm from [2, 3]. EigenPro converges very quickly in practice (usually within 10 iterations), as compared to 10k iterations for the non-pre-conditioned version in Eq. (11)).

Remarks. There are two important implementation details to be aware of when applying EigenPro in practice. First, it is important to note that estimating the top k eigenvectors and eigenvalues $\{(u_i, \lambda_i)\}_{i=1}^{k}$ can be done much more efficiently than performing a complete eigen-decomposition (as you will show in the homework), and as such, we can efficiently compute and use the the matrix B as a preconditioner. The second, more important, detail is that we will typically not be able to materialize the matrix K in memory as it is quadratic in the number of samples. (For reference, even a 100k x 100k matrix can take close to $40GB$ to store, and we cannot possibly store a 1 million x 1 million matrix.) Thus, we have to turn to mini-batching the update rule in Eq. (11). You will work through the details of this tweak in the homework.

References

[1] S. Axler. *Measure, integration & real analysis*. Springer Nature, 2020.

[2] S. Ma and M. Belkin. Diving into the shallows: a computational perspective on large-scale shallow learning. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017.

[3] S. Ma and M. Belkin. Kernel machines that adapt to GPUs for effective large batch training. In *Conference on Machine Learning and Systems*, 2019.